0.32.1. Le trajet dans un tableau — de base, ailleurs

Le trajet du 0.32 à partir de (10, 0), avec le B.

1. Ce qu'il faut comprendre

C'est le 0.32, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : le **départ (10, 0)** : les mêmes tableaux, le carré à droite.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.32.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <deplace>` → 0.25.3. La fonction deplace() — avancer sur les deux axes
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **constante (const)** — Une valeur qui a un nom et ne change jamais : « const uint8_t VIES = 3; ».
- **for** — Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **tableau** — Une rangée de boîtes sous un même nom : « uint8_t cases[10]; » donne cases[0] à cases[9].
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// Le trajet : un côté par case. PAS_X[i] et PAS_Y[i] vont ensemble.`	Un commentaire : une phrase pour celui qui lit (« Le trajet : un côté par case. PAS_X[i] et PAS_Y[i] vont ensemble. »). La console l'ignore.
`//  côté 0  côté 1  côté 2  côté 3`	Un commentaire : une phrase pour celui qui lit (« côté 0 côté 1 côté 2 côté 3 »). La console l'ignore.
`//  droite  bas  gauche  haut`	Un commentaire : une phrase pour celui qui lit (« droite bas gauche haut »). La console l'ignore.

`#include <deplace>  // fait avancer une tuile sur les deux axes`	Demande à la console « deplace » (ce que c'est : fait avancer une tuile sur les deux axes). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « fait avancer une tuile sur les deux axes ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`const uint8_t PAS_X[] = {  5,  0, -5,  0 };`	Un tableau constant nommé PAS_X : une liste de nombres, rangés dans l'ordre et gravés dans la cartouche. PAS_X[0] est le premier.
`const uint8_t PAS_Y[] = {  0,  5,  0, -5 };`	Un tableau constant nommé PAS_Y : une liste de nombres, rangés dans l'ordre et gravés dans la cartouche. PAS_Y[0] est le premier.
`uint8_t x = 10;  // la colonne du B`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 10. (Le commentaire le dit : « la colonne du B ».)
`uint8_t y = 0;  // sa ligne`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « sa ligne ».)
`int main() { // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`// Le changement : le B (ALPHABET[1]) part de (10, 0).`	Un commentaire : une phrase pour celui qui lit (« Le changement : le B (ALPHABET[1]) part de (10, 0). »). La console l'ignore.
`while (true) { // le trajet, sans fin`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « le trajet, sans fin ».)
`for (uint8_t i = 0; i < sizeof(PAS_X); i++) { // i = 0, 1, 2, 3 : un tour par côté`	Une boucle qui compte : i part de 0 ; on continue tant que i est plus petit que sizeof(PAS_X) ; et après chaque tour, i augmente de 1. (Le commentaire le dit : « i = 0, 1, 2, 3 : un tour par côté ».)
`deplace(x, y, ALPHABET[1], PAS_X[i], PAS_Y[i]);`	Appel de la fonction deplace de la console, avec les arguments x, y, ALPHABET[1], PAS_X[i], PAS_Y[i]. Ce qu'elle fait : les deux axes en une ligne : pasX pour la droite (+) ou la gauche (-), pasY pour le bas (+) ou le haut (-). 5 et 5 : en diagonale. À écrire seule sur sa ligne : deplace(x, y, ALPHABET[0], 5, 0); range la colonne dans x et la ligne dans y.
`// le côté i : ses deux pas lus dans`	Un commentaire : une phrase pour celui qui lit (« le côté i : ses deux pas lus dans »). La console l'ignore.
`// les tableaux. i = 0 : (5, 0), à droite ;`	Un commentaire : une phrase pour celui qui lit (« les tableaux. i = 0 : (5, 0), à droite ; »). La console l'ignore.
`// i = 1 : (0, 5), en bas ; etc.`	Un commentaire : une phrase pour celui qui lit (« i = 1 : (0, 5), en bas ; etc. »). La console l'ignore.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1196 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        63 o  ← la console, à l'allumage  — la mise en route de la console
  deplace()       236 o  ← main()  — une fonction fournie par le langage
  main()           67 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
  PAS_X_0           4 o  ← main()  — une table gravée
  PAS_Y_1           4 o  ← main()  — une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 2    26 o sur place, dans deplace()  + AdresseCase (10 o)
  effacer() × 1    15 o sur place, dans deplace()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 1     3 o sur place, dans deplace()  + AttendreImage (28 o)
Tes fonctions appelées :
  deplace() × 1    39 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le trajet : un côté par case. PAS_X[i] et PAS_Y[i] vont ensemble.
//                         côté 0   côté 1   côté 2   côté 3
//                         droite   bas      gauche   haut

#include <deplace>    // fait avancer une tuile sur les deux axes
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PAS_X

```
const uint8_t PAS_X[] = {  5,  0, -5,  0 };
```

3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PAS_Y

```
const uint8_t PAS_Y[] = {  0,  5,  0, -5 };
```

4. UNE VARIABLE — X

```
uint8_t x = 10;       // la colonne du B
```

5. UNE VARIABLE — Y

```
uint8_t y = 0;       // sa ligne
```

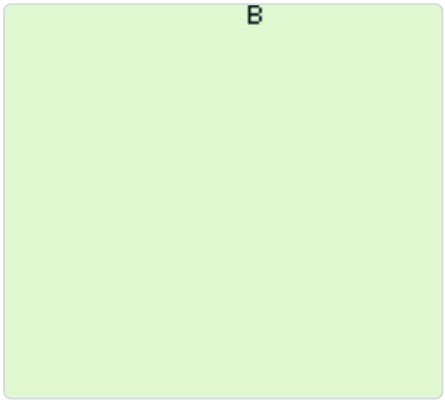
6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                        // Le programme commence ici.
  // Le changement : le B (ALPHABET[1]) part de (10, 0).
  while (true) {                                    // le trajet, sans fin
    for (uint8_t i = 0; i < sizeof(PAS_X); i++) {   // i = 0, 1, 2, 3 : un tour par côté
      deplace(x, y, ALPHABET[1], PAS_X[i], PAS_Y[i]);
                                                    // le côté i : ses deux pas lus dans
                                                    // les tableaux. i = 0 : (5, 0), à droite ;
                                                    // i = 1 : (0, 5), en bas ; etc.
    }
  }
}
```

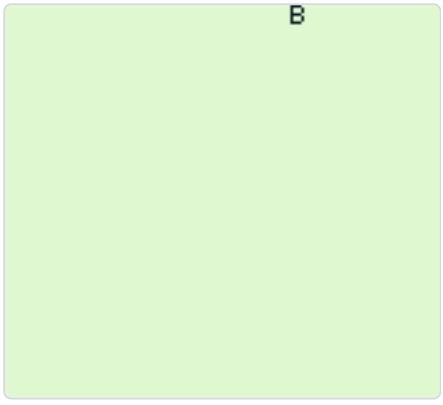
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

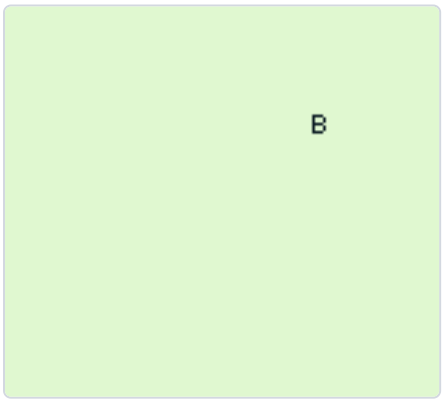
la toute première image — la console vient de démarrer — image 1



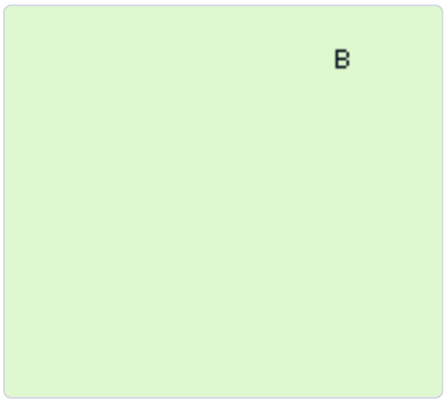
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Un B qui tourne en carré sans fin, à partir de (10, 0).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B passe par (15, 0)

✓ le B passe par (15, 5)

✓ le B passe par (10, 5)

Ensuite : **0.32.2. Le trajet dans un tableau — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++